TGC-MPD: Tree-Guided Confidence-Preserved Correction for Hard-Case Malicious npm Package Detection

1st Guosheng Tan
School of Computer Engineering
Jiangsu University of Technology
Changzhou, China
tanguosheng@jsut.edu.cn

2nd Yuhao Feng
School of Computer Engineering
Jiangsu University of Technology
Changzhou, China
2023352109@smail.jsut.edu.cn

3rd Yuxuan Zhou
School of Computer Engineering
Jiangsu University of Technology
Changzhou, China
zhouyx@jsut.edu.cn

4th Jiahui Li
School of Computer Engineering
Jiangsu University of Technology
Changzhou, China
2024145123@smail.jsut.edu.cn

5th Yiming Geng
School of Computer Engineering
Jiangsu University of Technology
Changzhou, China
gengyiming62@gmail.com

6th Ling Lu
School of Computer Engineering
Jiangsu University of Technology
Changzhou, China
yolo30878@gmail.com

***Abstract*—Malicious npm packages often share static risk signals with benign packages, including install scripts, confusing names, and unusual dependency patterns. This overlap makes suspicious packages difficult to distinguish, even with strong tabular detectors. We propose TGC-MPD, a tree-guided detector for hard-case malicious npm package detection. Its neural TreeCross branch combines raw static features, teacher risk priors, and random forest (RF) leaf embeddings to produce a correction score. A validation-selected gate retains the histogram gradient boosting (HGB) teacher score outside an uncertainty region and selects the correction score within it. We evaluate two NPMStudy-derived settings across three random seeds. Under the package-version protocol, TGC-MPD achieves the highest mean hard-case F1 and recall among the compared learned models. F1 increases from 0.9458 to 0.9535, and recall from 0.9310 to 0.9487, relative to fixed-threshold HGB. Ablations and paired cases support selective correction of ambiguous packages. The hard-case F1 gain is not observed under package-name-grouped testing, which measures transfer to unseen names.**

***Index Terms*—malicious package detection, software supply chain security, npm, open-source ecosystem, tree-guided learning, feature interaction, static analysis, confidence gate**

I. Introduction

Open-source package managers are now part of the security boundary of modern software. The npm ecosystem is a frequent target for three practical reasons. Package installation can run scripts, dependencies are deeply nested, and names or scopes can resemble legitimate projects. A malicious package does not need a memory-corruption exploit to cause harm. It can abuse install hooks, confusing names, encoded commands, or suspicious dependency layouts. Small source-level changes can also be hard to inspect manually. Package-level detection is therefore an important layer in software supply-chain defense [1]–[7].

Static screening can inspect packages before installation without executing attacker-controlled code. Prior detectors learn from metadata and static package features [5], [7], [8]. Our feature set covers manifest fields, script tokens, dependency statistics, and source layout. These signals are rarely decisive on their own. Benign packages may have install scripts, scoped names, or large dependency sets. Risk often appears in combinations. Examples include a scoped name with a confusing token, an install-time script with risky commands, or an unusual source layout with dependency anomalies.

Such combinations make malicious package detection a feature-interaction problem. Tree ensembles capture nonlinear rules and high-order interactions on tabular features [9]–[12]. Neural models also represent feature combinations [13]–[15]. A strong tree model can therefore provide both risk estimates and interaction structure for a correction branch. Our question is whether this guidance can improve hard suspicious cases while retaining teacher scores outside a selected uncertainty region.

We address this problem with TGC-MPD, a Tree-Guided Confidence-Preserved Malicious Package Detector. It targets hard cases in which benign and malicious packages share static risk signals. HGB supplies the primary risk score, while RF leaf identifiers encode learned feature combinations. A neural TreeCross branch combines these tree-derived signals with raw static features to produce a candidate correction score. A confidence gate selects that score within a validation-selected uncertainty region and retains the HGB score elsewhere.

Corresponding author: Guosheng Tan (tanguosheng@jsut.edu.cn).

This work was supported in part by the Jiangsu Province College Students' Network Security Innovation and Entrepreneurship Social Practice Project, in part by the Jiangsu Provincial Social Science Fund Project (Project No. 24DJD001), and in part by the Changzhou Soft Science Research Project (Project No. 20260535).

We evaluate TGC-MPD on NPMStudy-derived npm package data [4]. The scoped-balanced setting balances scoped and unscoped packages across benign and malicious labels. The hard-case setting increases overlap by selecting benign packages with suspicious static signals and malicious packages with weak or ambiguous cues. We compare against classical baselines, tree ensembles, and a neural MLP, with NPMStudy tool outputs as additional reference results. In the primary package-version evaluation, TGC-MPD improves hard-case F1 over HGB while retaining high performance on scoped-balanced packages. A separate package-name-grouped evaluation assesses transfer to unseen names.

This paper makes the following contributions:

- We study hard-case malicious npm package detection through a controlled setting that stresses overlapping benign and malicious static risk signals.
- We design TGC-MPD to combine raw static features, tree risk priors, and RF leaf embeddings with selective score replacement through a confidence gate.
- We demonstrate the highest mean hard-case F1 and recall among the compared learned models under package-version splits, supported by ablations and paired recovery cases.

Section II reviews the detection problem and related methods. Sections III and IV describe TGC-MPD and the evaluation protocol. Sections V–VII present results, discuss their scope, and conclude the paper.

II. Background and Related Work

A. Malicious npm Package Detection

Early ecosystem studies documented security risks in npm and package-manager supply chains [1]–[3]. Static and metadata-based detectors learn from package attributes [5], [7], [8]. DONAPI and Cerebro analyze behavior sequences [6], [16], while OSCAR and DySec use dynamic execution evidence [17], [18]. IntelliRadar extracts disclosed package intelligence [19]; Zhou et al. analyze malicious-package corpora and attack relationships [20]. PYPILINE combines suspicious-API knowledge with an agent workflow for PyPI detection [21]. NPMStudy supplies benchmark data and tool outputs [4]. These approaches establish useful package-level signals. Our focus is their overlap: suspicious packages can mix benign-looking and malicious-looking evidence, making them difficult to triage.

Malicious npm packages can expose risk across several static surfaces. Manifest fields reveal package names, scopes, versions, scripts, dependencies, repository links, licenses, and keywords. Install lifecycle scripts are especially important because they can run during installation. Source-layout features, including JavaScript file counts, JSON files, package size, and total source bytes, indicate whether a package resembles ordinary software or a thin wrapper. Name-confusion hints and risky script tokens add more security-specific signals.

The meaning of these features depends on context. Install scripts occur in benign packages, and a small source tree can contain malicious code. Combining script, dependency, and layout evidence allows a detector to learn conditional risk patterns.

B. Feature Interaction and Tabular Learning

DeepFM and DCN model feature interactions through factorization and cross networks [13]–[15]. TabNet uses feature attention, and TabR augments tabular learning with retrieval [22], [23]. Tree ensembles remain strong baselines for numerical tables [12], [24], [25]. TGC-MPD adapts established representation-learning ideas to overlapping package risks. It conditions a multilayer correction network on tree scores and learned leaf identifiers, then couples that representation with a validation-selected score gate. This integration connects tabular interaction structure to selective correction in malicious package detection.

C. Selective Score Correction

Global averaging combines predictor scores for every package. Selective correction instead replaces the default score only within a chosen region. This distinction motivates the gate in TGC-MPD and the global blending control in our experiments. It allows us to test whether concentrating correction on uncertain packages improves detection.

III. Method

Fig. 1 summarizes TGC-MPD. Static extraction captures pre-install evidence, the tree module supplies risk estimates and interaction regions, and TreeCross combines these signals to generate a correction score. The gate then controls which score is used. For example, the malicious sync-http-api/6.1.1 has a dependency-confusion hint but receives an HGB score of 0.259. The correction branch assigns 0.948, and the gate selects that score in seed 42, recovering the missed package (Table IV).

A. Static Risk Feature Extraction

Given an npm package archive, we extract static risk features from the manifest and package contents. The parser supports pre-install screening, where executing attacker-controlled install scripts is undesirable. The feature groups cover manifest metadata, script risk, source layout, and name-confusion risk. We remove columns that would create label leakage. These include malware-only archive size and labels derived from malicious-only annotations. The final model uses only numerical package-level features available through static parsing.

B. Tree Teacher

We train RF and HGB on the same static feature table. HGB supplies the primary teacher score based on its strong baseline performance. RF supplies auxiliary predictions and leaf assignments. The neural branch receives four risk priors: RF probability, HGB probability, their average, and their absolute disagreement. These inputs summarize the two models' risk estimates and differences.

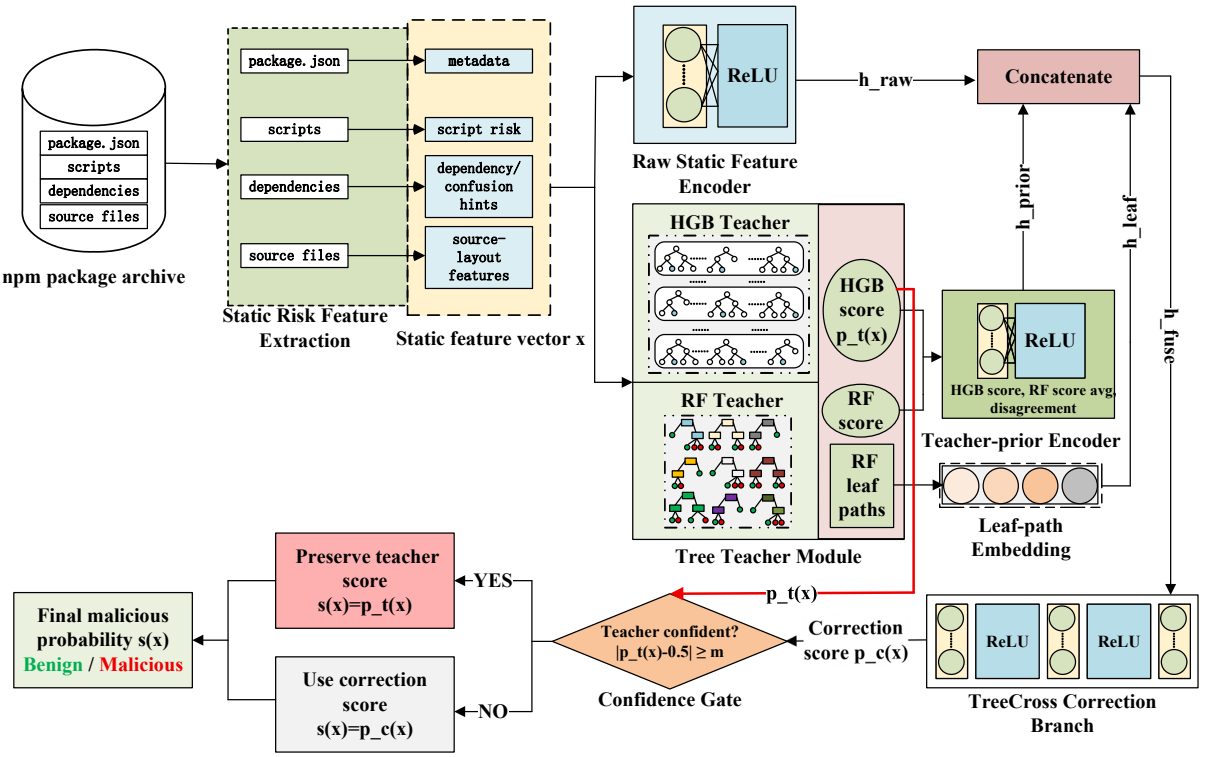


Fig. 1. Architecture of TGC-MPD.

C. Tree Leaf-Path Embedding

Tree leaf assignments encode high-order feature interactions discovered by the teacher. For each sample, the RF teacher maps the feature vector to one leaf per tree. We treat these leaf identifiers as categorical interaction tokens and learn an embedding for each observed leaf. The concatenated embeddings form a tree-guided interaction representation.

These embeddings let the neural branch use the RF's learned partitions alongside the original feature values.

D. TreeCross Correction Branch

The TreeCross branch combines standardized static features, four teacher priors, and embedded RF leaf identifiers. The raw encoder maps 40 features to 128 units. The prior encoder maps RF and HGB probabilities, their average, and their absolute disagreement to 32 units. Each RF leaf has an eight-dimensional learned embedding; concatenated embeddings are projected to 128 units. The fusion classifier has hidden widths 128 and 64 with ReLU activations and produces

$$s_c(x) = \sigma\big(f_c([h_{\text{raw}}; h_{\text{prior}}; h_{\text{leaf}}])\big), \tag{1}$$

where brackets denote concatenation and σ is the sigmoid function. We train with binary cross-entropy, dropout 0.2, and early stopping on validation F1.

E. Confidence-Preserved Gate

Let $s_t(x)$ be the teacher score and $s_c(x)$ be the TreeCross correction score, denoted by $p_t(x)$ and $p_c(x)$ in Fig. 1. Teacher confidence is measured as $|s_t(x) - 0.5|$. A validation-selected confidence margin m determines whether the final score preserves the teacher:

$$s(x) = \begin{cases} s_t(x), & |s_t(x) - 0.5| \geq m, \\ s_c(x), & |s_t(x) - 0.5| < m. \end{cases} \tag{2}$$

We search $m \in \{0, 0.05, \ldots, 0.50\}$ and, for each margin, classification thresholds $\tau \in \{0.05, 0.055, \ldots, 0.95\}$. The pair with the highest validation F1 is fixed before test evaluation; ties retain the first pair in ascending search order. The final label is $\hat{y} = \mathbb{1}[s(x) \geq \tau]$. The uncertainty region is centered at 0.5, while the classification boundary is τ. Teacher-score preservation therefore need not preserve labels when models use different thresholds. The released implementation evaluates both branches before selecting the output score.

IV. Experimental Setup

A. Dataset and Evaluation Settings

We use NPMStudy-derived package data. Malicious packages come from the NPMStudy archive. Benign packages are reconstructed from the npm registry. We build a random benign pool and parse package-level static features for both classes. Two balanced settings isolate different sources of static-signal overlap. Table I reports their composition: Scop.-B/M counts scoped benign/malicious packages, and Inst. B/M gives the fractions with install lifecycle scripts.

TABLE I
DATASET COMPOSITION BY EVALUATION SETTING.

Setting	Rows	Ben.	Mal.	Scop.-B	Scop.-M	Inst. B/M
scoped-balanced	2388	1194	1194	597	597	.221/.837
hard-case	1688	844	844	322	91	.457/.780

The scoped-balanced setting controls the scoped package indicator across labels. The hard-case setting stresses suspicious benign-malicious overlap rather than deployment prevalence. Benign hard cases are selected when they contain at least one suspicious static surface. Examples include an install lifecycle script, a risky script token, a dependency-confusion hint, or at least five package scripts. Malicious hard cases are selected when at least one common cue is weak or ambiguous. This includes no install lifecycle script, no risky install-script token, at most one package script, or a dependency-confusion hint. These criteria use only static package surfaces available to both classes. They do not use model predictions. The resulting split tests whether a detector can handle packages for which simple risk cues are less separable.

B. Baselines

We compare TGC-MPD with four reimplemented machine-learning baselines: logistic regression (LR), random forest (RF), histogram gradient boosting (HGB), and multilayer perceptron (MLP). GuardDog and SAP-XGB provide deterministic NPMStudy reference results [4]. Their available outputs cover 1,647 of 1,688 hard-case packages and 2,321 of 2,388 scoped-balanced packages across the full settings. These reference results use different evaluation subsets from the held-out tests of the learned models.

C. Metrics and Protocol

We report precision, recall, F1, and PR-AUC as the mean and sample standard deviation over seeds 40, 41, and 42. PR-AUC is computed as average precision, without trapezoidal interpolation. Package-version splits allocate 70%, 10%, and 20% to training, validation, and testing. Each run trains a 400-tree RF and an HGB teacher with 300 boosting iterations and learning rate 0.05. TreeCross uses AdamW with learning rate 10^{-3}, batch size 128, up to 160 epochs, and patience 24.

Table II uses fixed 0.5 thresholds for reimplemented baselines and a validation-selected threshold for TGC-MPD. Table III uses validation-selected thresholds for all variants. We additionally compare frozen HGB and gated scores at a common 0.5 threshold. Paired F1 uncertainty is assessed within each test split by 10,000 class-stratified bootstrap resamples of aligned predictions. Overlapping seed splits are not pooled as independent observations.

D. Deployment-Oriented Interpretation

TGC-MPD targets suspicious package review. Hard-case F1 and recall measure detection under overlapping static signals, while paired error transitions identify which teacher errors are recovered. The balanced setting controls ambiguity; precision at ecosystem prevalence requires separate evaluation.

V. RESULTS

A. Main Comparison

Fig. 2 and Table II report the primary package-version evaluation. TGC-MPD has the highest mean hard-case F1 and recall among the compared learned models. Relative to fixed-threshold HGB, F1 rises from 0.9458 to 0.9535 and recall from 0.9310 to 0.9487, with precision of 0.9587. This operating point recovers more malicious packages while retaining high precision. Fig. 2(a) shows mean F1 with one sample standard deviation; panel (b) shows the mean difference from fixed-threshold HGB. Tool reference results have no seed error bars. In Table II, bold numbers mark the best mean among learned models within each setting. The ablation below also compares TGC-MPD with a validation-tuned HGB teacher.

On scoped-balanced packages, TGC-MPD obtains 0.9668 F1, 0.9540 recall, and 0.9925 PR-AUC. HGB leads on F1 at 0.9724. The contrast with hard-case supports the intended focus on packages with more ambiguous static signals under this protocol.

B. Ablation Study

Table III evaluates all variants with validation-selected thresholds. The blending control also uses a validation-selected mixture weight. Bold values mark the best mean within each setting. Prior-only and leaf-only variants retain raw features but omit leaf inputs or prior inputs, respectively. The full ungated branch uses all three inputs. Adding the gate raises its hard-case F1 from 0.9443 to 0.9535. The complete model also exceeds the prior-only gate and validation-weighted HGB–TreeCross blend. Against the tuned HGB teacher, the mean F1 gain is 0.0037. The 0.0077 gain in Table II uses fixed-threshold HGB as the reference.

With a common final threshold of 0.5 applied to the frozen scores, hard-case F1 is 0.9486 for TGC-MPD and 0.9458 for HGB. This diagnostic retains the original validation-selected gate and shows a positive contribution from score replacement at the same final threshold.

C. Error Transition Analysis

On hard-case, seed 40 selects $m = 0$ and reproduces the tuned HGB decisions. Seeds 41 and 42 each correct four teacher errors and introduce two, increasing F1 by 0.0056 in each split. Their paired bootstrap 95% intervals for the F1 difference are $[-0.0094, 0.0203]$ and $[-0.0086, 0.0201]$. Both exact two-sided McNemar tests give $p = 0.6875$. These small paired changes support observed recovery cases, but do not establish statistical significance. On scoped-balanced, seeds 41 and 42 retain teacher decisions, while seed 40 fixes one error and introduces six. This contrast identifies where selective correction was useful in the present evaluation.

Table IV connects these error transitions to package features. Scores are malicious probabilities; predictions use each model's validation-selected threshold. The first two rows recover a malicious package without common script-risk cues and another with a dependency-confusion hint. The third corrects a false alarm on a scoped package with install

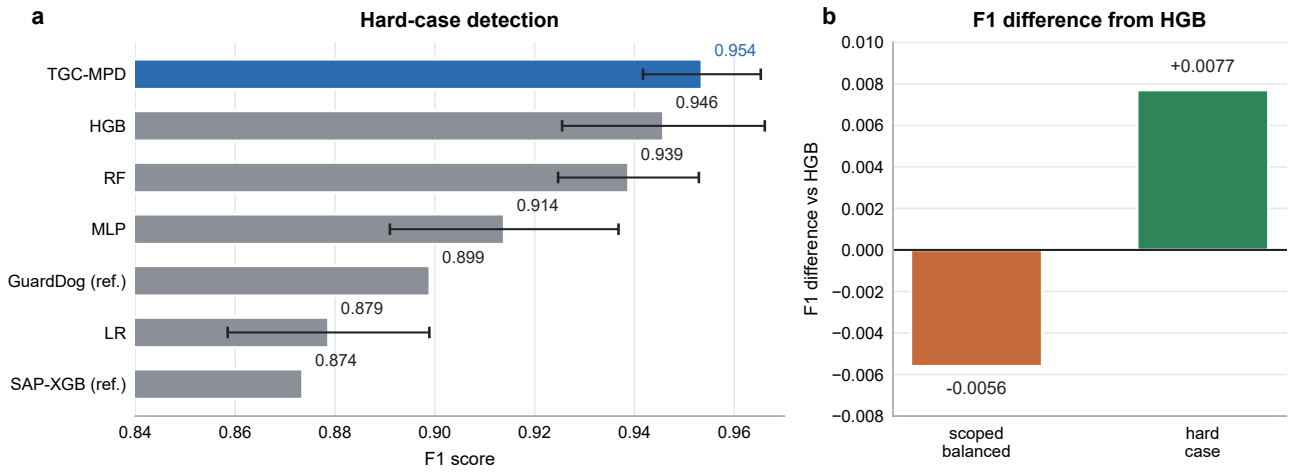


Fig. 2. Hard-case F1 and F1 differences from HGB.

TABLE II
DETECTION PERFORMANCE AND TOOL REFERENCE RESULTS.

Setting	Method	F1	Precision	Recall	PR-AUC
scoped-balanced	LR	0.9113±0.0088	0.8984±0.0043	0.9247±0.0151	0.9519±0.0109
	RF	0.9707±0.0069	**0.9942±0.0067**	0.9484±0.0087	0.9925±0.0020
	HGB	**0.9724±0.0113**	0.9870±0.0075	**0.9582±0.0151**	**0.9939±0.0016**
	MLP	0.9444±0.0077	0.9550±0.0240	0.9344±0.0097	0.9856±0.0013
	GuardDog (ref.)	0.9291	0.9485	0.9104	–
	SAP-XGB (ref.)	0.8799	0.9717	0.8040	–
	TGC-MPD	0.9668±0.0106	0.9803±0.0210	0.9540±0.0072	0.9925±0.0030
hard-case	LR	0.8787±0.0202	0.8779±0.0148	0.8797±0.0267	0.9245±0.0301
	RF	0.9388±0.0141	**0.9725±0.0133**	0.9073±0.0149	0.9872±0.0050
	HGB	0.9458±0.0203	0.9612±0.0132	0.9310±0.0280	**0.9880±0.0044**
	MLP	0.9139±0.0229	0.9188±0.0331	0.9093±0.0149	0.9568±0.0203
	GuardDog (ref.)	0.8990	0.9359	0.8649	–
	SAP-XGB (ref.)	0.8735	0.9837	0.7855	–
	TGC-MPD	**0.9535±0.0118**	0.9587±0.0264	**0.9487±0.0034**	0.9867±0.0034

TABLE III
F1 OF MODEL VARIANTS AND BLENDING CONTROLS.

Setting	Variant	F1
scoped-balanced	HGB teacher	**0.9702±0.0076**
	TreeCross prior	0.9633±0.0060
	TreeCross leaf	0.9614±0.0105
	TreeCross full	0.9689±0.0073
	HGB–full blend	0.9695±0.0084
	HGB–prior gate	0.9695±0.0085
	TGC-MPD gate	0.9668±0.0106
hard-case	HGB teacher	0.9498±0.0099
	TreeCross prior	0.9517±0.0100
	TreeCross leaf	0.9409±0.0023
	TreeCross full	0.9443±0.0197
	HGB–full blend	0.9522±0.0162
	HGB–prior gate	0.9519±0.0087
	TGC-MPD gate	**0.9535±0.0118**

scripts and many dependencies. The last row separates score preservation from classification: seed 42 retains 0.110, but the gated threshold of 0.105 flags the benign package, whereas the teacher threshold of 0.290 does not.

D. Package-Name Generalization

We repeat the evaluation with package-name isolation in both settings over seeds 40–42, giving six runs. In each run, ten stratified group folds are divided into seven training folds, one validation fold, and two test folds. All versions sharing a full npm package name, including its scope, remain in the same partition. No package name is shared between the training, validation, and test partitions within any run. With validation-selected thresholds for both models, hard-case F1 is 0.9007 ± 0.0115 for HGB and 0.8937 ± 0.0055 for TGC-MPD. Scoped-balanced F1 is 0.7399 ± 0.0049 and 0.7405 ± 0.0046, respectively. The hard-case gain is observed in the package-version evaluation but is not reproduced by this grouped test.

E. Computational Cost

A separate cost run used the original architecture and training settings on the saved hard-case seed-42 split. Measurements used an Intel Core i5-12400F with one CPU compute

TABLE IV
REPRESENTATIVE TEST CASES IN THE HARD-CASE SETTING.

Seed	Package	Label	HGB	TGC-MPD	Interpretation
41	chatbot-community/1.20.23	Mal.	0.325	0.773	This package has no install lifecycle script or risky script token. HGB missed it; the gated correction identified it as malicious.
42	sync-http-api/6.1.1	Mal.	0.259	0.948	HGB treated the package as benign despite a dependency-confusion hint; TGC-MPD raised the risk score and fixed the false negative.
41	@n3/pn-declaration/0.2.2	Ben.	0.611	0.157	A scoped benign package with install scripts and many dependencies was incorrectly flagged by HGB; the correction reduced the false positive.
42	orgtool-cli/0.2.1	Ben.	0.110	0.110	The score is preserved, but the lower classification threshold flags this benign package as malicious.

thread and an RTX 4060 Ti for CUDA operations. Feature-to-label inference includes scaling, teacher scoring, leaf mapping, neural evaluation, and gating; archive parsing and model loading are excluded. After five warm-ups, median times over 50 repetitions for 338 test packages were 11.16 ms for HGB, 75.91 ms for CPU TGC-MPD, and 73.61 ms with its neural branch on CUDA. CPU single-package latency was 2.07 ms and 34.75 ms, respectively. The median full-model batch time remains below 0.1 s on this workstation.

In separate processes using the same benchmark imports, sampled inference peak RSS was 493.3 MiB for HGB and 518.0 MiB for CPU TGC-MPD. CUDA TGC-MPD used 778.7 MiB host RSS and 14.0 MiB peak allocated GPU tensors, excluding CUDA context memory from the latter. HGB fitting took 1.40 s. Full model construction took 12.28 s, including RF fitting, representation preparation, CUDA branch training, and validation gate selection.

VI. Discussion

The package-version results connect the hard-case gain to selective correction. The complete model has higher mean F1 than the ungated branch and global blending control. The common-threshold comparison isolates the contribution of changed scores, while paired cases show which packages benefit. The gate and classification threshold jointly set the operating point.

In hard-case test splits, the gate selects correction scores for 7.4% and 3.8% of packages in seeds 41 and 42, and none in seed 40. Score replacement is thus concentrated on a small subset of packages in these runs. This makes the correction cases directly inspectable through paired teacher and model outputs.

A. Limitations

The balanced settings isolate static-signal overlap rather than ecosystem prevalence. Package-name isolation removes same-name version overlap but leaves publisher and code-family relationships uncontrolled. Three seeds and small paired error changes limit statistical conclusions. Temporal splits and naturally imbalanced cohorts would test performance under deployment conditions.

VII. Conclusion

This paper presented TGC-MPD for malicious npm packages with overlapping static risk signals. It combines raw features, tree risk priors, RF leaf embeddings, and a gate that controls score replacement. Under package-version splits, it achieved the highest mean hard-case F1 and recall among the compared learned models. Ablations and paired cases show how selective correction recovers teacher errors on ambiguous packages. These results support tree-guided correction for hard-case package review.

Data and Code Availability

The implementation and reproducibility scripts are available at https://github.com/GuoshengTan/TGC-MPD. Original datasets and package archives should be obtained from their original sources under the corresponding redistribution and safety policies.

Ethics Statement

This work analyzes malicious package artifacts for defensive research. The experiments use static features and do not execute package install scripts. Public release should avoid redistributing active malware payloads and should provide hashes, metadata, or controlled access where appropriate.

AI Usage Disclosure

AI assistance was used for drafting support, code organization, and experimental-result summarization. The authors remain responsible for the experimental design, result verification, claims, and final manuscript.

References

[1] M. Zimmermann, C.-A. Staicu, C. Tenny, and M. Pradel, "Small world with high risks: A study of security threats in the npm ecosystem," in *Proceedings of the 28th USENIX Security Symposium*, 2019, pp. 995–1010.

[2] R. Duan, O. Alrawi, R. P. Kasturi, R. Elder, B. Saltaformaggio, and W. Lee, "Towards measuring supply chain attacks on package managers for interpreted languages," in *Proceedings of the Network and Distributed System Security Symposium*, 2021.

[3] M. Ohm, H. Plate, A. Sykosch, and M. Meier, "Backstabber's knife collection: A review of open source software supply chain attacks," in *Proceedings of the 17th Conference on Detection of Intrusions and Malware, and Vulnerability Assessment*, ser. Lecture Notes in Computer Science, vol. 12223. Springer, 2020, pp. 23–43.

[4] W. Guo, Z. Chen, Z. Xu, C. Liu, M. Kang, S. Song, C. Liu, Y. Xu, W. Sun, and Y. Liu, "Understanding NPM malicious package detection: A benchmark-driven empirical analysis," 2026, arXiv preprint arXiv:2603.27549v1. [Online]. Available: https://arxiv.org/abs/2603.27549v1

[5] A. Sejfia and M. Schäfer, "Practical automated detection of malicious npm packages," in *Proceedings of the 44th International Conference on Software Engineering*, 2022, pp. 1681–1692.

[6] C. Huang, N. Wang, Z. Wang, S. Sun, L. Li, J. Chen, Q. Zhao, J. Han, Z. Yang, and L. Shi, "DONAPI: Malicious NPM packages detector using behavior sequence knowledge mapping," in *Proceedings of the 33rd USENIX Security Symposium*. USENIX Association, 2024, pp. 3765–3782.

[7] S. Halder, M. Bewong, A. Mahboubi, Y. Jiang, M. R. Islam, M. Z. Islam, R. H. L. Ip, M. E. Ahmed, G. S. Ramachandran, and M. A. Babar, "Malicious package detection using metadata information," in *Proceedings of the ACM Web Conference 2024*, 2024, pp. 1779–1789.

[8] H. Samaana, D. E. Costa, E. Shihab, and A. Abdellatif, "A machine learning-based approach for detecting malicious PyPI packages," in *Proceedings of the 40th ACM/SIGAPP Symposium on Applied Computing*, 2025, pp. 1617–1626.

[9] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[10] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[11] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, "CatBoost: Unbiased boosting with categorical features," in *Advances in Neural Information Processing Systems*, vol. 31, 2018.

[12] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on typical tabular data?" in *Advances in Neural Information Processing Systems*, vol. 35, 2022.

[13] H. Guo, R. Tang, Y. Ye, Z. Li, and X. He, "DeepFM: A factorization-machine based neural network for CTR prediction," in *Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence*, 2017, pp. 1725–1731.

[14] R. Wang, B. Fu, G. Fu, and M. Wang, "Deep & Cross Network for ad click predictions," in *Proceedings of the ADKDD'17*, 2017, pp. 1–7.

[15] R. Wang, R. Shivanna, D. Cheng, S. Jain, D. Lin, L. Hong, and E. Chi, "DCN V2: Improved deep & cross network and practical lessons for web-scale learning to rank systems," in *Proceedings of the Web Conference 2021*, 2021, pp. 1785–1797.

[16] J. Zhang, K. Huang, Y. Huang, B. Chen, R. Wang, C. Wang, and X. Peng, "Killing two birds with one stone: Malicious package detection in NPM and PyPI using a single model of malicious behavior sequence," *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 4, pp. 104:1–104:28, 2025.

[17] X. Zheng, C. Wei, S. Wang, Y. Zhao, P. Gao, Y. Zhang, K. Wang, and H. Wang, "Towards robust detection of open source software supply chain poisoning attacks in industry environments," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 2024, pp. 1990–2001.

[18] S. T. Mehedi, C. Islam, G. Ramachandran, and R. Jurdak, "DySec: A machine learning-based dynamic analysis for detecting malicious packages in PyPI ecosystem," 2025, arXiv preprint arXiv:2503.00324. [Online]. Available: https://arxiv.org/abs/2503.00324

[19] W. Guo, C. Liu, L. Wang, Y. Zhang, J. Wu, Z. Xu, and Y. Liu, "IntelliRadar: A comprehensive platform to pinpoint malicious package information from cyber intelligence," 2025, arXiv preprint arXiv:2409.15049v6. [Online]. Available: https://arxiv.org/abs/2409.15049v6

[20] X. Zhou, Y. Zhang, W. Niu, J. Liu, H. Wang, and Q. Li, "An analysis of malicious packages in open-source software in the wild," in *2025 55th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, 2025, pp. 371–385.

[21] S. Pang, Y. Yao, Z. Jiang, Z. Fan, H. Li, and B. Liu, "PYPILINE: Malicious PyPI package detection via suspicious API knowledge and agent workflow," 2026, arXiv preprint arXiv:2606.19063v3. [Online]. Available: https://arxiv.org/abs/2606.19063v3

[22] S. Ö. Arik and T. Pfister, "TabNet: Attentive interpretable tabular learning," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, no. 8, 2021, pp. 6679–6687.

[23] Y. Gorishniy, I. Rubachev, N. Kartashev, D. Shlenskii, A. Kotelnikov, and A. Babenko, "TabR: Tabular deep learning meets nearest neighbors," in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2024/hash/4ef594af0d9a519db8fb292452c461fa-Abstract-Conference.html

[24] R. Shwartz-Ziv and A. Armon, "Tabular data: Deep learning is not all you need," *Information Fusion*, vol. 81, pp. 84–90, 2022.

[25] V. Borisov, T. Leemann, K. Seßler, J. Haug, M. Pawelczyk, and G. Kasneci, "Deep neural networks and tabular data: A survey," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 35, no. 6, pp. 7499–7519, 2024.